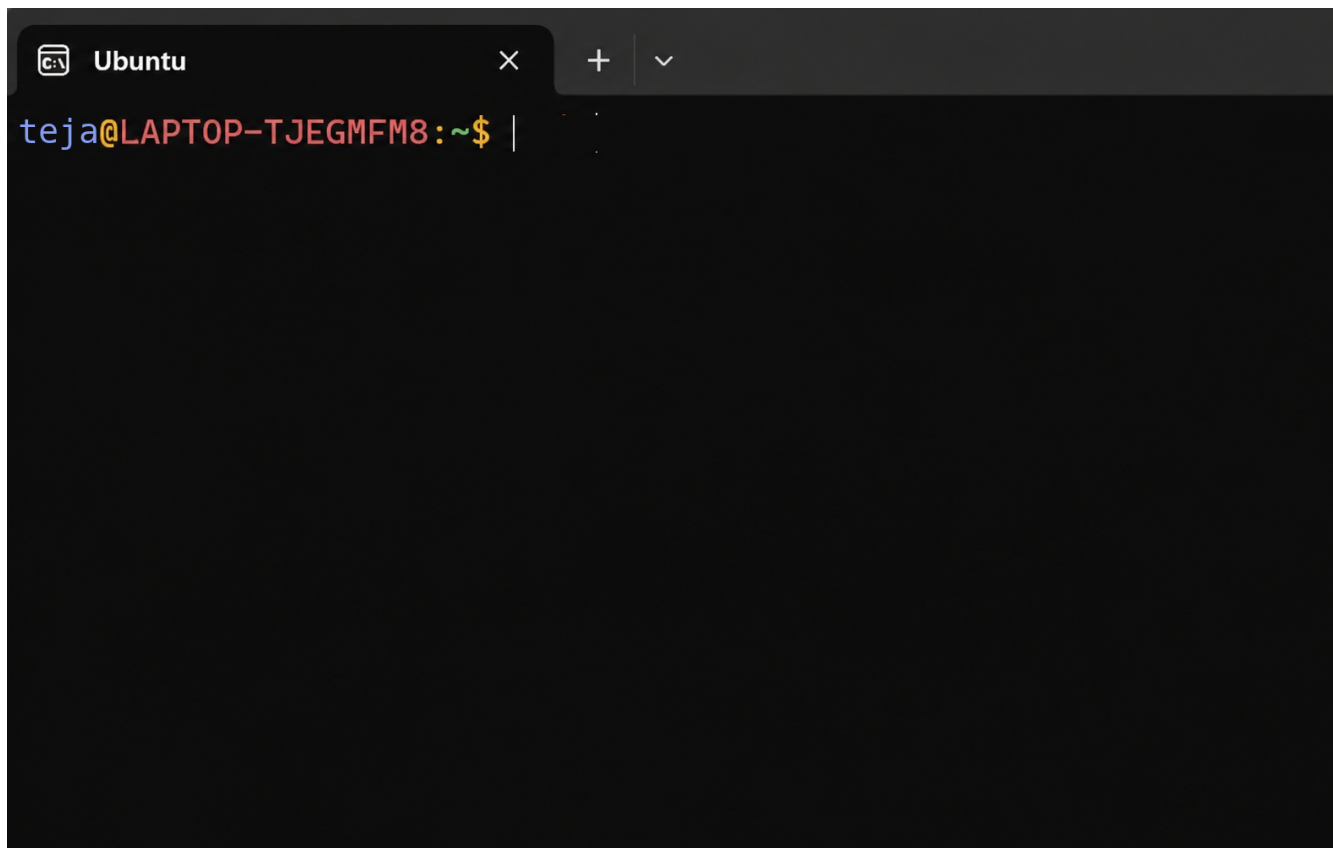


OSSP – WEEK-1 –PRACTICAL

To Develop a c program that demonstrates how a Linux O.S executes a command entered by the user using fork(),exec(), and wait() system calls.

Step 1: Open UBUNTU terminal

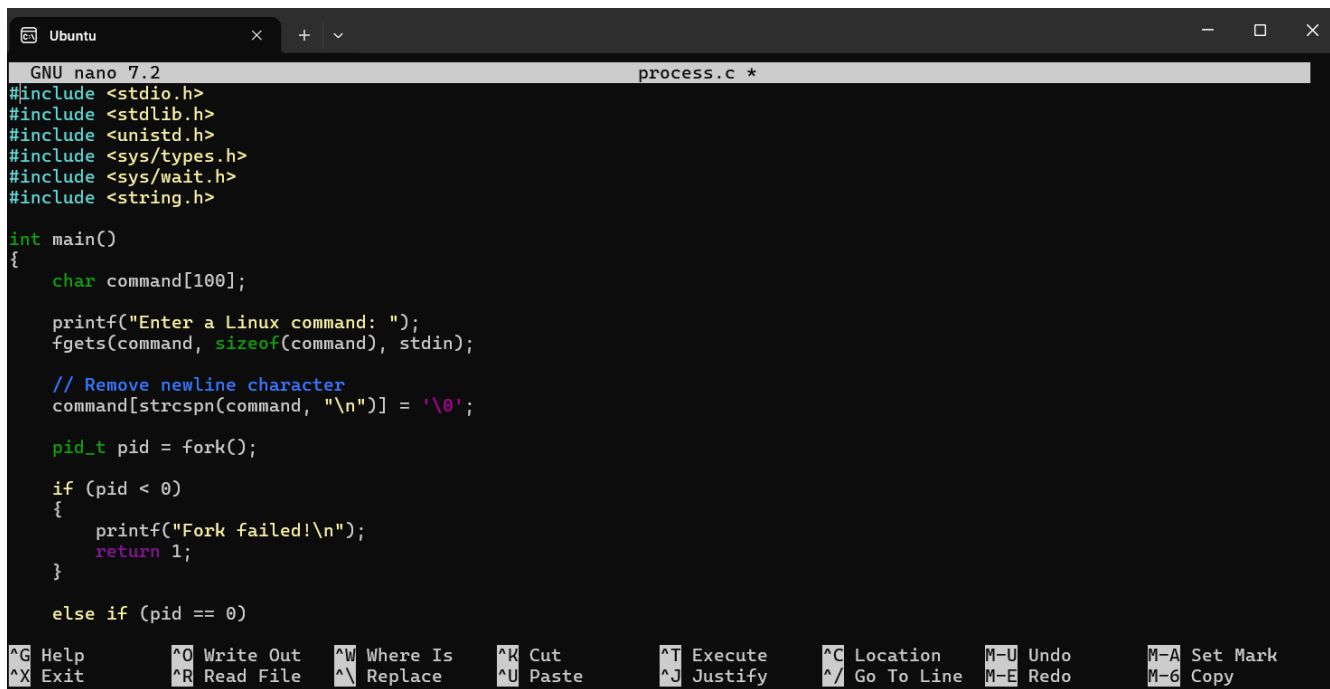
You will get something like:



Step 2: Create the C file

Type: `nano process.c`

Press Enter.



```
GNU nano 7.2 process.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0)
    {
        // Child process
        // Execute the command
        system(command);
    }

    // Parent process
    wait(NULL);

    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^_ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy

Step 3: Paste the program

Paste the complete C program into nano.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];
    char *args[20];
    char *token;
    int i = 0;

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    token = strtok(command, " ");

    while (token != NULL && i < 19)
    {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;

    printf("Parent PID: %d\n", getpid());

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());

        execvp(args[0], args);

        perror("exec failed");
        exit(1);
    }
    else
    {
        wait(NULL);
        printf("Parent: Child process completed.\n");
    }

    return 0;
}

```

Then press:

Ctrl + O

You will see:

```
File Name to Write: process.c
```

Press Enter.

Then press:

Ctrl + X

You are back at the terminal.

Step 4: Compile the program

Type:

```
gcc process.c -o process
```

Press Enter.

If there is no error message, compilation was successful.

Step 5: Run the program

Type:

```
./process
```

You will see:

```
Enter a Linux command:
```

Step 6: Enter a command

For the first demonstration, type:

```
ls
```

Press Enter.

```
teja@LAPTOP-TJEGMFM8:~$ nano process.c
teja@LAPTOP-TJEGMFM8:~$ gcc process.c -o process
teja@LAPTOP-TJEGMFM8:~$ ./process
Enter a Linux command: ls

Parent Process
Parent PID : 99
Child PID  : 100

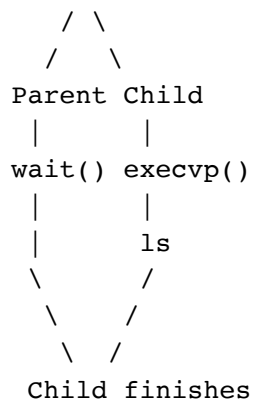
Child Process
Child PID : 100
Parent PID: 99
chown_case fork_exec fork_exec.c memory_case process process.c

Child process completed.
teja@LAPTOP-TJEGMFM8:~$ |
```

Parent: Child process completed.

What happened?

```
./process
|
v
Parent process
|
fork()
```

Step 7: Try another command

Run again:

```
./process
```

Enter:

```
Pwd
```

```
teja@LAPTOP-TJEGMFM8:~$ ./process
Enter a Linux command: pwd

Parent Process
Parent PID : 107
Child PID  : 108

Child Process
Child PID : 108
Parent PID: 107
/home/teja

Child process completed.
teja@LAPTOP-TJEGMFM8:~$ |
```

Step 8: Try ls -l

Run:

```
./process
```

Enter:

```
ls -l
```

This demonstrates that your program can accept command + arguments.

```
teja@LAPTOP-TJEGMFM8:~$ ./process
```

```
Enter a Linux command: ls -l
```

```
Parent Process
```

```
Child Process
```

```
Child PID : 113
```

```
Parent PID: 112
```

```
Parent PID : 112
```

```
Child PID  : 113
```

```
Command execution failed!
```

```
Child process completed.
```

```
teja@LAPTOP-TJEGMFM8:~$ |
```

Step 9: Try uname

Run:

```
./process
```

Enter:

```
Uname
```

Step 10: strace — useful for demonstration

Since this practical is about showing how Linux executes processes, you can also demonstrate it with strace.

First install it if necessary:

```
sudo apt install strace
```

Then:

```
strace ./process
```

When it asks for the command, enter:

```
ls
```

You'll see many system calls such as:

```
fork()  
execve()  
wait4()
```

This is useful to show your faculty because it gives evidence of the system calls being used.

For the record:

strace is a Linux utility used to trace system calls and signals made by a process. It can be used to observe calls such as fork(), execve(), and wait4() during process creation and command execution.

LINUX HARDWARE & OS SERVICES

Now they asked us to investigate:

```
uname  
lscpu  
lsblk  
ps  
top
```

and prepare a report explaining how Linux abstracts:

- CPU
- Memory
- Storage
- I/O devices

1. uname

Command

```
uname -a
```

```
teja@LAPTOP-TJEGMFM8:~$ uname -a
Linux LAPTOP-TJEGMFM8 5.10.102.1-microsoft-standard-WSL2 #1 SMP Wed Mar 2 00:30:59 UTC 2022 x86_64 x86_64 x86_64 GNU/Linux
```

2. lscpu

Command

```
lscpu
```

```
teja@LAPTOP-TJEGMFM8:~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                39 bits physical, 48 bits virtual
Byte Order:                   Little Endian
CPU(s):                       12
  On-line CPU(s) list:        0-11
Vendor ID:                    GenuineIntel
Model name:                   12th Gen Intel(R) Core(TM) i5-1235U
CPU family:                   6
Model:                        154
Thread(s) per core:           2
Core(s) per socket:           6
Socket(s):                    1
Stepping:                     4
BogoMIPS:                     4992.01
Flags:                         fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse ss
                               e2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop
                               _tsc cpuid pni pclmulqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline
                               _timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch invpcid_single ssbd ibrs
                               ibpb stibp ibrs_enhanced tpr_shadow vnmi ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi
                               2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves umip waitp
                               kg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_clear serialize flush_l1d arch_capabili
                               ties
Virtualization features:
  Virtualization:              VT-x
  Hypervisor vendor:           Microsoft
  Virtualization type:         full
Caches (sum of all):
  L1d:                         288 KiB (6 instances)
  L1i:                         192 KiB (6 instances)
  L2:                           7.5 MiB (6 instances)
  L3:                           12 MiB (1 instance)
Vulnerabilities:
  Itlb multihit:                Not affected
  L1tf:                         Not affected
  Mds:                           Not affected
  Meltdown:                     Not affected
  Spec store bypass:            Mitigation; Speculative Store Bypass disabled via prctl and seccomp
  Spectre v1:                   Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2:                   Mitigation; Enhanced IBRS, IBPB conditional, RSB filling
  Srbds:                        Not affected
  Tsx async abort:              Not affected
teja@LAPTOP-TJEGMFM8:~$
```


3. lsblk

Command

lsblk

```
teja@LAPTOP-TJEGMFM8:~$ lsblk
NAME
    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0 256G  0 disk
sdb      8:16   0 256G  0 disk /
teja@LAPTOP-TJEGMFM8:~$ |
```

4. ps

Command

Ps

```
127 teja@LAPTOP-TJEGMFM8:~$ ps
PID TTY          TIME CMD
  69 pts/0        00:00:00 bash
 151 pts/0        00:00:00 ps
127 teja@LAPTOP-TJEGMFM8:~$ |
```

5. top

Command

top

To exit top, press: q

```
top - 04:42:45 up 9 min,  0 user,  load average: 0.00, 0.00, 0.00
Tasks:  5 total,   1 running,  4 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7784.2 total,   7571.4 free,    286.4 used,   110.0 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,     0.0 used.  7497.8 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	1804	1180	1104	S	0.0	0.0	0:00.02	init
67	root	20	0	1824	80	0	S	0.0	0.0	0:00.00	init
68	root	20	0	1824	96	0	S	0.0	0.0	0:00.05	init
69	teja	20	0	6068	5260	3532	S	0.0	0.1	0:00.06	bash
155	teja	20	0	9300	5188	3028	R	0.0	0.1	0:00.00	top

Important: top does not finish by itself. It continuously monitors the system until you press q.